

a. Lagrange interpolation

```
In [1]: def lagrange_interpolation(x, x_known, f_known):
        """
        Evaluate the Lagrange interpolating polynomial at x.

        x: point to evaluate
        x_known: list of known x-coordinates (nodes)
        f_known: list of known function values at x_known
        """

        # polynomial degree = points - 1
        n = len(x_known) - 1

        result = 0.0

        # loop over L_j(x)
        for j in range(n + 1):
            pj = 1.0

            # loop over all other nodes k!=j
            for k in range(n + 1):
                if k != j:
                    pj *= (x - x_known[k]) / (x_known[j] - x_known[k])

            result += f_known[j] * pj

        return result
```

```
In [2]: #test data
xi = [0.0, 0.4, 0.8, 1.2, 1.6]
fi = [0.0, 0.428392, 0.742101, 0.910314, 0.970348]

exact = {0.3: 0.328627, 0.5: 0.520500}

# loop over the 2 test points
for i in (0.3, 0.5):
    # compute interpolant
    interpol = lagrange_interpolation(i, xi, fi)
    # compare to the known exact value
    err = interpol - exact[i]
    # print results
    print(f"f({i}) interpolated = {interpol:.6f};    exact = {exact[i]:.6f};    error = {err:.6f}")
```

```
f(0.3) interpolated = 0.329345;    exact = 0.328627;    error = 0.000718
f(0.5) interpolated = 0.519939;    exact = 0.520500;    error = -0.000561
```

b. Aitken vs Lagrange

```
In [3]: def lagrange_interpolation_counted(x, x_known, f_known):
        """
        Evaluate the Lagrange interpolating polynomial at x, but also counts
        arithmetic operations.

        x: point to evaluate
        x_known: list of known x-coordinates (nodes)
        f_known: list of known function values at x_known
        """

        # polynomial degree = points - 1
        n = len(x_known) - 1

        ops = {'add': 0, 'sub': 0, 'mul': 0, 'div': 0} # to count the
        addition, subtraction, multiplication and division
        result = 0.0

        # loop over L_j(x)
        for j in range(n + 1):
            pj = 1.0

            # loop over all other nodes k != j
            for k in range(n + 1):
                if k != j:
                    num = x - x_known[k]
                    ops['sub'] += 1

                    den = x_known[j] - x_known[k]
                    ops['sub'] += 1

                    pj *= num / den
                    ops['div'] += 1
                    ops['mul'] += 1

            result += f_known[j] * pj
            ops['mul'] += 1
            ops['add'] += 1

        return result, ops
```

```

In [ ]: def aitken_interpolation_counted(x, x_known, f_known):
        """
        Aitken/Neville recursive interpolation, counting arithmetic
        operations.

        x: point to evaluate
        x_known: list of known x-coordinates (nodes)
        f_known: list of known function values at x_known
        """

        # polynomial degree = points - 1
        n = len(x_known) - 1

        ops = {'add': 0, 'sub': 0, 'mul': 0, 'div': 0} # to count the
        addition, subtraction, multiplication and division
        ft = list(f_known)

        # loop over triangular table level i
        for i in range(n):
            for j in range(n - i):
                a = x - x_known[j]
                ops['sub'] += 1

                b = x_known[i+j+1] - x_known[j]
                ops['sub'] += 1

                c = x - x_known[i+j+1]
                ops['sub'] += 1

                term1 = (a/b) * ft[j+1]
                ops['div'] += 1
                ops['mul'] += 1

                term2 = (-c/b) * ft[j]
                ops['div'] += 1
                ops['mul'] += 1

                ft[j] = term1 + term2
                ops['add'] += 1

        return ft[0], ops

```

```

In [ ]: # print header row
print(f"{'n':>4} {'pts':>5} {'Lagrange':>10} {'Aitken':>10}
{'Aitken/Lagrange':>16}")

# try more problem sizes n (polynomial degree) to see how the
# operation counts scale – denser at small n, extended further at large
n
for n in (2, 4, 6, 8, 12, 16, 20, 30, 40, 60, 80, 100, 150, 200):

    # n+1 equally-spaced nodes from [0, 1]
    x_known = []
    for i in range(n + 1):
        x_known.append(i / n)

    # test function f(x) = 1/(1+x^2) at those nodes
    f_known = []
    for xx in x_known:
        f_known.append(1 / (1 + xx**2))

    # fixed evaluation point
    x_test = 0.37

    # run both algorithms and get (value, op-count)
    v1, o1 = lagrange_interpolation_counted(x_test, x_known, f_known)
    v2, o2 = aitken_interpolation_counted(x_test, x_known, f_known)

    # merge op-counts into 1
    opL, opA = sum(o1.values()), sum(o2.values())

    print(f"{'n':4d} {'n+1':5d} {'opL':10d} {'opA':10d} {'opA/opL':14.2f}
match={abs(v1-v2)<1e-9}")

```

n	pts	Lagrange	Aitken	Aitken/Lagrange	
2	3	30	24	0.80	match=True
4	5	90	80	0.89	match=True
6	7	182	168	0.92	match=True
8	9	306	288	0.94	match=True
12	13	650	624	0.96	match=True
16	17	1122	1088	0.97	match=True
20	21	1722	1680	0.98	match=True
30	31	3782	3720	0.98	match=True
40	41	6642	6560	0.99	match=True
60	61	14762	14640	0.99	match=True
80	81	26082	25920	0.99	match=True
100	101	40602	40400	1.00	match=True
150	151	90902	90600	1.00	match=True
200	201	161202	160800	1.00	match=True

c. Rounding error in direct Aitken

```
In [ ]: import numpy as np
        from fractions import Fraction as F
        import scipy as sp
```

```
In [ ]: def lagrange_interpolation_f32(x, x_known, f_known):
        """
        Direct Lagrange interpolation.

        x: point to evaluate
        x_known: list of known x-coordinates (nodes)
        f_known: list of known function values at x_known
        """

        # polynomial degree = points - 1
        n = len(x_known) - 1

        # cast everything to float32 to make rounding error accumulate
        # faster
        x = np.float32(x)
        s = np.float32(0.0)

        x_known_f32 = []
        for v in x_known:
            x_known_f32.append(np.float32(v))
        x_known = x_known_f32

        f_known_f32 = []
        for v in f_known:
            f_known_f32.append(np.float32(v))
        f_known = f_known_f32

        # loop over L_j(x)
        for j in range(n + 1):
            p = np.float32(1.0)

            # loop over all other nodes k != j
            for k in range(n + 1):
                if k != j:
                    # (x - x_known[k]) / (x_known[j] - x_known[k])
                    p = np.float32(p * ((x - x_known[k]) / (x_known[j] -
                    x_known[k])))

            # add contribution
            s = np.float32(s + f_known[j] * p)

        # convert back to a regular Python float for the return value
        return float(s)
```

```

In [ ]: def aitken_interpolation_f32(x, x_known, f_known):
        """
        Direct Aitken interpolation, forced to single-precision arithmetic
        to make rounding effects visible at moderate n.

        x: point to evaluate
        x_known: list of known x-coordinates (nodes)
        f_known: list of known function values at x_known
        """

        # polynomial degree = points - 1
        n = len(x_known) - 1

        # cast everything to float32 to make rounding error accumulate
        # faster
        x = np.float32(x)

        x_known_f32 = []
        for v in x_known:
            x_known_f32.append(np.float32(v))
        x_known = x_known_f32

        ft_f32 = []
        for v in f_known:
            ft_f32.append(np.float32(v))
        ft = ft_f32

        # loop triangular table one level i
        for i in range(n):
            for j in range(n - i):
                # combine x_known[j] with ft[j+1], weighted by how close x
                # is to x_known[j]
                term1 = np.float32((x - x_known[j]) / (x_known[i+j+1] -
                x_known[j]) * ft[j+1])

                # combine x_known[i+j+1] with ft[j], weighted by how close x
                # is to x_known[i+j+1]
                term2 = np.float32((x - x_known[i+j+1]) / (x_known[j] -
                x_known[i+j+1]) * ft[j])

                # overwrite ft[j]
                ft[j] = np.float32(term1 + term2)

        # convert back to a regular Python float for the return value
        return float(ft[0])

```

```
In [ ]: def lagrange_interpolation_exact(x, x_known, f_known):
        """
        Lagrange_interpolation, but using exact rational

        x: point to evaluate (Fraction)
        x_known: list of known x-coordinates (Fractions)
        f_known: list of known function values at x_known (Fractions)
        """

        # polynomial degree = points - 1
        n = len(x_known) - 1      s = F(0)

        # loop over L_j(x)
        for j in range(n + 1):
            p = F(1)
            # loop over all other nodes k != j
            for k in range(n + 1):
                if k != j:
                    p *= (x - x_known[k]) / (x_known[j] - x_known[k])
            s += f_known[j] * p
        return s
```

```
In [13]: # header row
print(f"{'n':>4} {'Lagrange err':>14} {'Aitken err':>12}
{'Aitken/Lagrange':>16}")

# test with increasing numbers of points to watch rounding error grow
for n in (15, 18, 20, 24):

    # n+1 equally-spaced nodes in [0, 2]
    x_known = np.linspace(0.0, 2.0, n + 1)

    # f(x) = J0(x) - the Bessel function
    f_known = sp.jv(0, x_known)

    x_test = 0.9 # same as book's example

    exact = lagrange_interpolation(x_test, x_known, f_known)

    # evaluated at the same x
    lag32 = lagrange_interpolation_f32(x_test, x_known, f_known)
    ait32 = aitken_interpolation_f32(x_test, x_known, f_known)

    # absolute error of each candidate against the exact reference
    err_lag, err_ait = abs(lag32 - exact), abs(ait32 - exact)

    # print n, both errors, and how many times bigger Aitken's error is
    print(f"{'n':4d} {'err_lag':14.3e} {'err_ait':12.3e}
{'err_ait/err_lag':16.2f}")
```

n	Lagrange err	Aitken err	Aitken/Lagrange
---	--------------	------------	-----------------

NameError Traceback (most recent call last)

Cell In[13], line 11

```
7 # n+1 equally-spaced nodes in [0, 2]
8 x_known = np.linspace(0.0, 2.0, n + 1)
9
10 # f(x) = J0(x) - the Bessel function
---> 11 f_known = sp.jv(0, x_known)
12
13 x_test = 0.9 # same as book's example
14
```

NameError: name 'sp' is not defined

In []:

Exported with [runcell](#) — convert notebooks to HTML or PDF anytime at [runcell.dev](#).